COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 DATA STRUCTURES AND ALGORITHMS	**SECTION:**	CPE21S3
DATE PERFORMED:	JULY 9 2026	**DATE SUBMITTED:**	JULY 11 2026
NAME:	DUSTIN CARL B. MAPAS	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 1.1_ BASIC C++ PROGRAMMING

1. PROCEDURE OUTPUT

ILO A:

Code:

```cpp
#include <iostream>

class Operations {
    public:
        int sumOfTwoNum(int fNum, int sNum);
        const bool greaterThan(int fNum, int sNum);
};

const bool logicalOp(bool fNum, bool sNum);
```

```cpp
int main(){
    Operations num;
    std::cout<<"the sum of the two numbers is: " <<num.sumOfTwoNum( fNum: 6, sNum: 7)<<std::endl;
    std::cout<<std::endl;
    num.greaterThan( fNum: 6, sNum: 7);
    std::cout<<std::endl;
    logicalOp( fNum: 1, sNum: 1);

    return 0;
}
```

```
21
22  int Operations::sumOfTwoNum(int fNum, int sNum){
23      int sum = fNum + sNum;
24      return sum;
25  }
26
27  const bool Operations::greaterThan(int fNum, int sNum){
28      bool value = fNum > sNum;
29      std::cout << "Is variable A greater than B? " << std::boolalpha << value << std::endl;
30      return true;
31  }
32
```

```
33  const bool logicalOp(bool fNum, bool sNum){
34      bool bit_And = fNum && sNum;
35      bool bit_Or = fNum || sNum;
36      bool bit_Xor = fNum ^ sNum;
37      bool bit_Not = !fNum;
38      bool bit_Not2 = !sNum;
39      std::cout << "Results are: " << std::boolalpha << "Logical"<<"\n AND: "
40      << bit_And << " \n OR: " << bit_Or << "\n Xor: "
41      << bit_Xor << " \n NOT: " << "\n for variable 1: " << bit_Not
42      <<"\n for variable 2: "<< bit_Not2<< std::endl;
43      return true;
44  }
```

Output:

```
C:\Users\Dread\Desktop\hoa1.1-20260712T0
the sum of the two numbers is: 13

Is variable A greater than B? false

Results are: Logical
 AND: true
 OR: true
 Xor: false
 NOT:
 for variable 1: false
 for variable 2: false

Process finished with exit code 0
```

Code Analysis:
In this task my header file declaration is #include <iostream> and then I created a class called Operations that has two public methods which are sumOfTwoNum which has a data type of int, and another method called greaterThan which has a const bool data type and both parameters has fNum, and sNum as int. Next is that I defined each function under the main driving code, both sumOfTwoNum, and greaterThan has a class scope definition since these functions belong to the class Operations while the logicalOp function is not since it was declared globally. After defining the needed functions, I began creating the main program, first I created a variable named num with a class data type of Operations and then I displayed the necessary task like the sum of the two numbers, is the first number greater than the second one, and the logical operations of the two numbers. Lastly, while calling the functions that belong to a class, I used the dot operator wherein num. And then followed by the function gives me access to that function that belongs to that class.

Sections	**Answer**
Header File Declaration Section	`#include <iostream>`
Global Declaration	`const bool logicalOp(bool fNum, bool sNum);`

Class Declaration and Method Definition Section	`class Operations {`
	`  public:`
	`  int sumOfTwoNum(int fNum, int sNum);`
	`  const bool greaterThan(int fNum, int sNum);`
	`};`

Main Function	`int main(){`
	`  Operations num;`
	`  std::cout<<"the sum of the two numbers is: "`
	`<<num.sumOfTwoNum(6,7)<<std::endl;`
	`  std::cout<<std::endl;`
	`  num.greaterThan(6,7);`
	`  std::cout<<std::endl;`
	`  logicalOp(1,1);`
	
	`  return 0;`
	`}`

Method Definition	`nt Operations::sumOfTwoNum(int fNum, int sNum){`
	`  int sum = fNum + sNum;`
	`  return sum;`
	`}`

```
const bool
Operations::greaterThan(int fNum, int
sNum){
   bool value = fNum > sNum;
   std::cout << "Is variable A
greater than B? " << std::boolalpha
<< value << std::endl;
   return true;
}

const bool logicalOp(bool fNum, bool
sNum){
   bool bit_And = fNum && sNum;
   bool bit_Or = fNum || sNum;
   bool bit_Xor = fNum ^ sNum;
   bool bit_Not = !fNum;
   bool bit_Not2 = !sNum;
   std::cout << "Results are: " <<
std::boolalpha << "Logical"<<"\n AND:
" << bit_And << " \n OR: " << bit_Or
<< "\n Xor: " << bit_Xor << " \n NOT:
" << "\n for variable 1: " << bit_Not
<<"\n for variable 2: "<< bit_Not2<<
std::endl;
   return true;
}
```

Table 1-1. C++ Structure Code for Answer

ILO B:

Code:

```
#include <iostream>

class Triangle{
    private:
        double totalAngle, angleA, angleB, angleC;
    public:
        //constructor
        Triangle(double A, double B, double C);
        void setAngles(double A, double B, double C);
        const bool validateTriangle();
};

```

```cpp
int main(){

    Triangle setl( A: 40, B: 30, C: 110);

    if(setl.validateTriangle()){
        std::cout<<"The shape is valid triangle \n";
    }
    else{
    std::cout<<"Not a triangle";
     }
    return 0;
}
```

```cpp

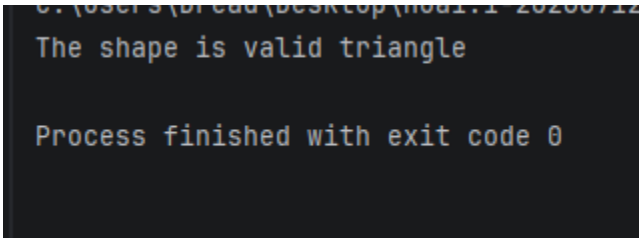
Triangle::Triangle(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;

}

void Triangle::setAngles(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;
}

const bool Triangle::validateTriangle(){
    return(totalAngle <= 180);
}
```

Output:



Output observation, and comments:
The output displayed that the shape is indeed a triangle since setl has the values of each sides; 40, 30, and 110. During the if-else statement the validateTriangle function has been called with the corresponding value since it was called with the help of the dot operator. This function validates the shape by adding all the angles, and if the total angle is less than or equal to 180 it is a valid triangle.

Table 1-2. ILO B output observations and comments.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. Create a C++ program to swap the two numbers in different variables.

```cpp
#include <iostream>
//supplementary #1
void swapNum(double &fNum, double &sNum){
    double temp = fNum;
    fNum = sNum;
    sNum = temp;
}

int main(){
    double fNum,sNum;
    std::cout<<"Enter Value A: ";
    std::cin>>fNum;
    std::cout<<"Enter Value B: ";
    std::cin>>sNum;
    std::cout<<"Value A: "<<fNum<<"\t Value B:"<<sNum<<std::endl;

    std::cout<<"Swapped Values: "<<std::endl;
    swapNum([&]fNum,[&]sNum);
    std::cout<<"Value A: "<<fNum<<"\t Value B:"<<sNum<<std::endl;

    return 0;
}
```

Output:

```
C:\Users\Dread\Desktop\hoa1.1-202607
Enter Value A:20

Enter Value B:10

Value A: 20      Value B:10
Swapped Values:
Value A: 10      Value B:20

Process finished with exit code 0
```

Code Analysis:

First I created a function called swapNum with a void data type and then defined it. It has a parameter of a double data type and a reference symbol with the variable name fNum and sNum. Inside the function definition, I created a variable temp with a double data type which stores the value of fNum, and then fNum will have the value of sNum, while sNum will have the value of temp. Thus swapping both values with the help of the reference operator the swap

in the main code will be instantly reflected. Moving on to the main code, the program will ask for the first and second value and then it will call the function and the swapping will begin. After that, it will display the swapped values.

2. Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.

```cpp
#include <iostream>
//supplementary #2
double convTemp(double kelvin){
    double fahrenheit = 1.8 * (kelvin - 273) + 32;
    return fahrenheit;

}
int main(){
    double kelvin;
    std::cout<<"Kelvin to Fahrenheit"<<std::endl;
    std::cout<<"Enter temperature in Kelvin: ";
    std::cin>>kelvin;
    std::cout<<"Kelvin: "<<kelvin<< "\nFahrenheit: " << convTemp(kelvin)<< std::endl;

    return 0;
}
```

Output:

```
"C:\Users\Dread\Desktop\hoa1.1-2026
Kelvin to Fahrenheit
Enter temperature in Kelvin:600

Kelvin: 600
Fahrenheit: 620.6

Process finished with exit code 0
```

Code Analysis:

In this task I created a function for converting kelvin to fahrenheit. The function has a double data type and has a parameter called kelvin and will return fahrenheit after converting kelvin to fahrenheit. In the main code, I declared variable kelvin and then display what conversion is happening and then the program will ask for the temperature in kelvin and after inputting it, the program calls the function and then displays the converted temperature to fahrenheit.

3. Create a C++ program that has a function that will calculate the distance between two points.

```cpp
#include <iostream>
#include <cmath>
double distance_Calc( double x1, double y1, double x2, double y2 );

int main () {
double x1, y1, x2, y2;

    std::cout<<"Enter x1: ";
    std::cin>>x1;
    std::cout<<"Enter y1: ";
    std::cin>>y1;
    std::cout<<"Enter x2: ";
    std::cin>>x2;
    std::cout<<"Enter y2: ";
    std::cin>>y2;

    std::cout<<"The distance between the two points is: " << distance_Calc( x1, y1, x2, y2 );

    return 0;
}

double distance_Calc( double x1, double y1, double x2, double y2 ) {

    double distance = sqrt( std::pow((x2-x1),2) + std::pow((y2-y1),2 ));
    return distance;
}
```

Output:

```
"C:\Users\Dread\Desktop\hoa1.1-20260712T030758Z-2
Enter x1:10

Enter y1:10

Enter x2:20

Enter y2:20

The distance between the two points is: 14.1421
Process finished with exit code 0
```

Code Analysis:

In this task I include the cmath library for the distance formula and then I created a function prototype wherein it calculates the distance between two points. Next is that I defined that function, I declared distance as a double and the distance formula is assigned to that variable, then it will return the distance. Lastly, the main code will ask for the user to enter the points, and after entering all points, the function distance_Calc will be called and then displays the

distance between two points.

4. Modify the code given in ILO B and add the following functions:
a. A function to compute for the area of a triangle
b. A function to compute for the perimeter of a triangle
c. A function that determines whether the triangle is acute-angled, obtuse-angled or 'others.'

```
#include <iostream>

class Triangle{
private:
    double totalAngle, angleA, angleB, angleC;
public:
    //constructor
    Triangle(double A, double B, double C);
    void setAngles(double A, double B, double C);
    const bool validateTriangle();
    double getArea(double base, double height);
    double Perimeter(double A, double B, double C);
    const char angleType();
};

```

```
int main(){
    std::cout<<"The shape has the sides of 40, 30, and 110"<<std::endl;
    Triangle setl( A: 40, B: 30, C: 110);

    if(setl.validateTriangle()){
        std::cout<<"The shape is a valid triangle \n";
        std::cout<<"The perimeter of the triangle is: "<<setl.Perimeter( A: 40, B: 30, C: 110)<<std::endl;
        std::cout<<"The area of the triangle with base: 40 and height: 30 is: "<<setl.getArea( base: 40, height: 30)<<std::endl;
        std::cout<<"This triangle is an " << setl.angleType()<<std::endl;
    }
    else{
        std::cout<<"Not a triangle";
    }
    return 0;
}
```

```
31
32  Triangle::Triangle(double A, double B, double C){
33      angleA = A;
34      angleB = B;
35      angleC = C;
36      totalAngle = A+B+C;
37
38  }
39
40  void Triangle::setAngles(double A, double B, double C){
41      angleA = A;
42      angleB = B;
43      angleC = C;
44      totalAngle = A+B+C;
45  }
46
47  const bool Triangle::validateTriangle(){
48      return(totalAngle == 180);
49  }
```

```
50
51  double Triangle::getArea(double base, double height) {
52      double area = 0.5 * (base*height);
53      return area;
54  }
55
56  double Triangle::Perimeter(double A, double B, double C) {
57      double perimeter = A+B+C;
58      return perimeter;
59  }
60
61  const char Triangle::angleType() {
62      if (angleA == 90 or angleB == 90 or angleC == 90) {
63          std::cout<<"Right Angle"<<std::endl;
64      }
65      else if ( angleA > 90 or angleB > 90 or angleC > 90 ) {
66          std::cout<<"Obtuse Angle"<<std::endl;
67      }
68      else{ std::cout<<"Acute Angle"<<std::endl;}
69
70  }
```

Output:

```
C:\Users\Dread\Desktop\hoa1.1-20260712T030758Z-2-001\hoa1.1\supp4(2).exe
The shape has the sides of 40, 30, and 110
The shape is a valid triangle
The perimeter of the triangle is: 180
The area of the triangle with base: 40 and height: 30 is: 600
This triangle is an Obtuse Angle

Process finished with exit code -1073741795 (0xC000001D)
```

Code Analysis:

First the code has a class named Triangle wherein it has private objects, and public methods. In this supplementary activity I added the function getArea, Perimeter, and angleType inside the class. The function for calculating the area is called getArea and it is defined by using the area of a triangle formula that is assigned to the variable area and will return the area. Next is the Perimeter function, it calculates the perimeter of the triangle by adding all the sides of the triangle. Lastly is the angleType function, this identifies if the triangle is acute, obtuse, or right. It is defined by an if else statement so that if any of the angles is at 90 it will be a right triangle, if it is more than 90, it is an obtuse, else it is an acute. Moving on to the main code, it will display that the shape has the sides of 40, 30, and 110. I created a variable called “setl” with the class data type of Triangle, and then an if-else statement so that if the shape is a triangle it will display that the shape is a triangle, it will calculate the area, perimeter, and identify the type of the triangle. Otherwise it will display that it is not a triangle.

Conclusion:

In conclusion, I learned a lot of concepts, and this activity guided me to understand c++ better. Moreover, this activity gave me the chance to recall a lot of concepts like class, and reference operator. Furthermore, I did an analysis for the procedure, and supplementary activity. With that, I think I did well in this activity, although it was a lot of work, it did help me familiarize myself with coding again and even the concepts. Lastly, I strive to be better, and enhance my programming skills.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

COMPUTER ENGINEERING

QUEZON CITY

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts and codes**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used gemini as a tool for understanding. I wanted to know how some of the code lines worked, and how a specific code affects the program.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.